MORA · BACKEND · CRITÉRIO DE AVALIAÇÃO

O backend utiliza migrações de dados de maneira profissional

Como o projeto se adequou ao critério · Peso 1.0 · Outubro de 2026 · branch `feat/migracoes` (PR para a `develop`)

> **Rubrica.** Mínimo: o banco de dados é criado através de ferramenta adequada (ex. Liquibase). Níveis acima: evidência de migrações criadas em pelo menos 30% e, para 100%, em pelo menos 50% das sprints.

6 de 6	3	0	3 de 6
serviços com banco usam ferramenta de migração	ferramentas: Flyway, Umzug, node-pg-migrate	scripts para aplicar à mão	sprints com migrações versionadas

1. Onde estávamos

Serviço	Como o banco era criado	Atendia?
comunicacao, portaria, plan, meeting	Hibernate `ddl-auto: update`, mais SQL avulso aplicado à mão em cada máquina	Não
auth-api	Funções próprias rodando a cada boot, dependentes de tabelas criadas pelo `init-databases.sql`	Não
financeiro	`node-pg-migrate`, mas só por comando manual; no Docker ninguém rodava	Em parte

Na prática, o esquema nascia de dois mecanismos concorrentes (o script de inicialização do Postgres e o Hibernate), e o que nenhum dos dois fazia ficava para scripts que cada um lembrava, ou não, de rodar.

2. O que fizemos

Cada serviço passou a criar e evoluir o próprio banco com uma ferramenta de migração, **executada automaticamente quando ele sobe**, antes de atender requisições:

Serviço	Banco	Ferramenta	Histórico no banco
comunicacao-service	`mora`	Flyway	`flyway_historico_comunicacao`
portaria-service	`mora`	Flyway	`flyway_historico_portaria`
plan-service	`mora_plan`	Flyway	`flyway_historico_plan`
meeting-service	`mora_meeting`	Flyway	`flyway_historico_meeting`
auth-api	`auth_db`	Umzug (executor de migrações do Sequelize)	`SequelizeMeta`
financeiro	`mora_financeiro`	node-pg-migrate	`pgmigrations`

O gestao-geral não tem banco. O `init-databases.sql` agora só cria os bancos vazios.

Serviços Java: Flyway com o Hibernate em `validate`

- Migrações versionadas em `src/main/resources/db/migration` (`V1__estrutura_inicial.sql`, `V2__...`).
- O Hibernate deixou de criar tabelas: com `ddl-auto: validate` ele só confere, e o serviço não sobe se a entidade pedir uma coluna que o banco não tem.
- **O V1 não foi transcrito à mão.** Cada serviço subiu contra um banco vazio, a estrutura foi lida do catálogo do Postgres e convertida em SQL, e o resultado foi aplicado noutro banco vazio e comparado coluna a coluna: **zero diferenças** nos quatro serviços.
- O V2 traz o que o Hibernate não cria e que vinha de scripts manuais: índices por condomínio, índices parciais e regras (CHECK, `ON DELETE CASCADE`).

auth-api: Umzug

As 13 migrações que já existiam passaram a rodar uma vez por banco, na ordem certa, com o nome gravado em `SequelizeMeta`. A primeira cria as tabelas que vinham do script de inicialização. Um teste automatizado protege a numeração e a ordem das que dependem umas das outras.

financeiro: node-pg-migrate na subida

As migrações já eram versionadas; agora o próprio serviço as aplica antes de escutar a porta. Se uma falhar, ele não sobe, e o erro diz o motivo.

3. Banco novo e banco existente

A troca não podia quebrar o ambiente de quem já rodava o projeto. O desenho cobre três casos:

- **Banco novo:** tudo é criado pelas migrações.
- **Banco existente:** o V1 é idempotente (`IF NOT EXISTS`), não muda nada e só entra no histórico.
- **Banco atrasado:** o V1 cria as colunas que a entidade ganhou e o banco ainda não tem.

O banco `mora` é dividido entre comunicacao e portaria. Por isso cada serviço tem a própria tabela de histórico, e o baseline é na versão 0, para que o segundo serviço a subir num banco novo não pule o próprio V1.

4. O que a adoção revelou e corrigiu

Defeito encontrado	Efeito
Três tabelas do portaria (`blocos`, `areas_comuns`, `funcionario_jornadas`) não eram criadas num banco novo: a regra de unicidade citava `condominioId` sem aspas	A escala de funcionários quebrava em instalação nova; as outras duas só existiam porque o script de inicialização as criava
O CHECK de `avisos.publico_alvo` não tinha o valor `SINDICO`, que entrou no código em agosto	Publicar aviso só para o síndico falhava. Corrigido pela migração V3 do comunicacao
`poll.data_hora_fim` faltava no banco em uso	Detectado pelo `validate`; agora a migração cria a coluna
Índices e regras existiam só em quem tinha rodado scripts à mão	Agora vão a todo banco pelo V2
O script de inicialização inseria um administrador com senha fixa	Removido; a conta inicial vem de variável de ambiente

5. Como foi verificado

Com as imagens Docker desta versão, os seis serviços subiram em ambientes isolados, cada um com um Postgres próprio:

Cenário	Resultado
Volume vazio	**Os seis criaram o próprio esquema e subiram, com o Hibernate validando**
Cópia do banco em uso	**Os seis subiram; só o que faltava foi criado (coluna atrasada, CHECK corrigido)**
Segunda subida	**"No migration necessary" e "Banco em dia"**
Testes automatizados	**auth-api 449 e financeiro 65, todos passando**

6. Evidência por sprint

O cronograma tem 6 sprints (17/03 a 15/06/2026). Pelo histórico do Git, o código que hoje roda como migração foi criado em:

Sprint	O que entrou	Hoje é
S2 (05 a 13/04)	Tabelas do `auth_db` e do `mora` em SQL versionado	Migração `001-estrutura-inicial` do auth-api e V1 do portaria
S3 (26/04 a 04/05)	Seed das estruturas físicas	Migração de dados (ressalva: é seed, não esquema)
S4 (23/05 a 04/06)	`migrate-rf03`, `migrate-rf07`, `migrate-condominios`	Migrações 002, 003, 004 e 007 do auth-api
S5 (até 15/06)	`migrate-portaria`	Migração 013 do auth-api

São **3 de 6 sprints (50%)** sem contar o seed, e 4 de 6 contando. Depois das sprints, de agosto a outubro, entraram mais de 20 migrações.

Ressalva: durante as sprints essas migrações rodavam sem ferramenta de mercado. Elas eram versionadas no Git desde a sprint 2 e foram padronizadas em Flyway, Umzug e node-pg-migrate. A partir de agora, toda mudança de esquema é uma migração nova.

7. Como verificar

```
# banco novo: apagar o volume e subir (atenção: apaga os dados locais)
docker compose -f docker/docker-compose.yml down -v
docker compose -f docker/docker-compose.yml up -d --build

# histórico de cada ferramenta
docker exec postgres psql -U admin -d mora -c "select version, description from
flyway_historico_portaria"
docker exec postgres psql -U admin -d auth_db -c 'select name from "SequelizeMeta"'
docker exec postgres psql -U admin -d mora_financeiro -c "select name from pgmigrations"
```

Documentação completa no repositório: `docs/MIGRACOES.md`.
